Studio Nx

Backend(FastAPI/Python) 05 — decode_access_token() · _decode_token()

2026-10-07

Backend(FastAPI/Python) 05 — `decode_access_token()` · `_decode_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-03 백엔드 공통 — 토큰 검사 · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	04 `unexpected error handler()` · 06 `get current user()`

1. 하는 일

요청에 실려 온 Access Token 이 **우리 서버가 만든 진짜 토큰이고, 아직 유효한지** 확인한 뒤, 토큰 주인의 `user_id` 를 돌려준다. 문제가 있으면 401 오류를 던진다.

`decode_access_token("eyJhbGciOi…")` → `7` (회원 번호)

함수	역할
`decode_access_token()`	Access Token 검사. 바깥에서 부르는 이름
`_decode_token()`	실제 검사. P1-05 의 `decode_refresh_token()` 도 같은 함수를 쓴다 (`type` 만 다름)

2. 언제 쓰나

로그인이 필요한 **모든 API 요청마다** 한 번씩 불린다.

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard` + 헤더 `Authorization: Bearer eyJhbGciOi…` (JS 02)
② `get_current_user()` (다음 06)	헤더에서 토큰 글자를 꺼내 **이 함수**에 넘김
③ 이 함수	검사 → `7`
④ `get_current_user()`	DB 에서 7번 회원을 찾아 라우터에 넘김
실패하면	401 `UNAUTHORIZED` → 브라우저의 `api.js` 가 리프레시 쿠키로 새 토큰을 받고 한 번 다시 보냄

JWT(토큰)의 생김새

토큰은 점(`.`)으로 나뉜 세 부분이다.

부분	예	내용
머리	`eyJhbGciOiJIUzI1NiJ9`	서명 방식 `{"alg": "HS256"}`
내용	`eyJzdWIiOiI3Ii…`	`{"sub": "7", "type": "access", "exp": 1791273600}`
서명	`kQ3x…`	머리 + 내용을 **서버 비밀값(`JWT_SECRET`)**으로 계산한 값

칸	뜻
`sub`	토큰 주인 — 회원 번호 (글자로 저장)

칸	뜻
`type`	`access` (30분, API 요청용) 또는 `refresh` (3일, 재발급용)
`exp`	만료 시각 (1970년부터 센 초)

내용은 암호화되지 않는다. 누구나 풀어서 읽을 수 있다. 그래서 비밀번호 같은 것은 넣지 않는다. 대신 **서명** 덕분에 내용을 바꿀 수 없다 — `sub` 를 `7` 에서 `1` 로 바꾸면 서명이 맞지 않고, 비밀값을 모르는 사람은 맞는 서명을 만들 수 없다.

3. 코드

import (파일 맨 위)

```
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다.

import	가져오는 것	쓰는 곳
`jwt`	PyJWT 라이브러리 (`requirements.txt` 의 `pyjwt`)	`jwt.decode` · `jwt.PyJWTError`
`get_settings`	`.env` 를 읽은 설정 (`app/core/config.py`)	`get_settings().jwt_secret`
`AppError` · `ErrorCode`	01 의 오류 클래스 · 코드 (`app/core/errors.py`)	401 `UNAUTHORIZED`

설치 이름(`pyjwt`)과 import 이름(`jwt`)이 다르다.

함수

```
JWT_ALGORITHM = "HS256"


def decode_access_token(token: str) -> int:
    return _decode_token(token, "access")


def _decode_token(token: str, expected_type: str) -> int:
    invalid = AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    try:
        payload = jwt.decode(
            token,
            get_settings().jwt_secret,
            algorithms=[JWT_ALGORITHM],
            options={"require": ["exp", "sub", "type"]},
        )
    except jwt.PyJWTError:
        raise invalid from None
    if payload["type"] != expected_type:
        raise invalid
    try:
        return int(payload["sub"])
```

```
    except (TypeError, ValueError):
        raise invalid from None
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `jwt.decode`)
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)
`app.…`	우리 프로젝트(`studionx/backend/app/`)의 파일. `app.core.errors` = `app/core/errors.py`

식	뜻
`jwt.decode(token, 비밀값, algorithms=[…])`	PyJWT 라이브러리. 서명 확인 + 만료(`exp`) 확인 후 내용을 사전으로 돌려줌. 하나라도 틀리면 오류
`options={"require": [...]}`	이 칸들이 없으면 실패
`except jwt.PyJWTError:`	PyJWT 의 모든 오류(만료 `ExpiredSignatureError`, 서명 틀림 `InvalidSignatureError`, 형식 틀림 `DecodeError` …)의 부모. 한 번에 잡는다
`raise invalid from None`	우리 오류로 바꿔 던짐. `from None` 은 "원래 오류는 이어 붙이지 말라"는 뜻 — 로그가 짧아진다
`int(payload["sub"])`	`"7"` → `7`. `"admin"` 이면 `ValueError`, 숫자가 아닌 종류면 `TypeError`
`_decode_token` 의 `_`	이 파일 안에서만 쓰는 함수라는 파이썬 관례

토큰	결과
정상 (`sub: "7"`, `type: access`, 만료 전)	`7`
만료됨	401
다른 비밀값으로 서명	401
내용만 바꿈 (`sub` 7 → 1)	401 (서명 불일치)
`type: refresh`	401
`sub` 또는 `type` · `exp` 없음	401
`sub: "admin"`	401
서명 없는 토큰 (`alg: none`)	401
`"not-a-token"` · `""`	401

4. 흐름

1. 서명 · 만료 · 필수 칸을 확인한다 (`jwt.decode`). 실패하면 401.
2. `type` 이 `access` 인지 확인한다. 아니면 401.
3. `sub` 를 정수로 바꿔 돌려준다. 안 되면 401.

5. 왜 이렇게 (보안)

선택	이유
`algorithms` 를 `HS256` 하나로 고정	토큰 머리의 `alg` 는 보낸 사람이 마음대로 쓸 수 있다. `"none"`(서명 없음)을 믿으면 누구나 `sub: 1` 토큰을 만들 수 있다. 서버가 정한 방식만 받는다

선택	이유
`type` 확인	리프레시 토큰(3일)을 Access Token 자리에 넣어 30분 제한을 피하지 못하게 한다
실패 이유와 상관없이 같은 401 · 같은 문구	만료인지 위조인지 알려 주면 공격자에게 힌트가 된다. 프론트는 어차피 같은 처리(재발급)를 한다
`"Please sign in again."` 의 `UNAUTHORIZED`	프론트 `api.js` 가 재발급을 시도하는 조건과 같은 코드 (JS 02)
검사 부분을 `_decode_token()` 으로 분리	P1-05 `decode_refresh_token()` 은 `return _decode_token(token, "refresh")` 한 줄이 된다
비밀값은 `.env` 의 `JWT_SECRET`	코드 · Git 에 넣지 않는다. 32바이트 이상이어야 한다 (짧으면 PyJWT 가 경고). 지금 로컬 값은 64바이트 — 값은 출력하지 않고 길이만 확인

비밀값이 새면 누구나 아무 회원의 토큰을 만들 수 있다. 그때는 `JWT_SECRET` 을 바꾸면 기존 토큰이 모두 무효가 된다 (모든 사용자가 다시 로그인).

6. 시험 (`tests/test_security.py`, 새로 만듦 · 9개)

토큰 발급(`create_access_token`)은 P1-05 라서 시험에서 `jwt.encode` 로 직접 만들었다.

경우	결과
정상 토큰	`7`
만료 · 다른 비밀값 · `type: refresh`	401 `UNAUTHORIZED` "Please sign in again."
`sub` · `type` · `exp` 중 하나 없음 (3개)	401
`sub: "admin"`	401
`"not-a-token"` · 빈 글자	401
서명 없는 토큰 (`alg: none`)	401
내용만 바꾼 토큰	401

전체 시험 35개 통과, `ruff` 검사 통과.